tree with n elements

Operation	k-d tree	R-tree	Ss-tree
Search	O(log(n))	O(n)	O(n)
Insert	O(log(n))	O(n)	O(n)
Remove	$O(n^{1-1/k})^*$	O(n)	O(n)
nearestNeighbor	$O(2^k + \log(n))^*$	O(n)	O(n)
pointsInRegion	O(n)	O(n)	O(n)

(*) Amortized, for a k-d tree holding k-dimensional points.

Neither R-trees nor Ss-trees can offer logarithmic worst-case upper bounds for their methods: in the (unlikely, but possible) worst case, all leaves of the tree will be traversed, and there is at most n/m of them. This means that each of the main operations on these data structures can take up to linear time in the size of the dataset. Table 10.1 summarize their running time, comparing them to k-d tree's.

10.5.2 Mitigating Hyper-Spheres Limitations

Now the question arises: is there anything we can do to limit the cons of using spherical bounding envelopes, so that we can reap the advantages when we have symmetrical datasets, and limit the disadvantages with skewed ones?

To cope with skewed datasets, we could use ellipsoids instead of spheres, so that the clusters can grow in each direction independently. However, this would complicate search, as we would to compute the radius along the